

## BOOK 02 — JAVA OOPS

### Beginner to 10+ Years Interview Mastery (Telugu + English)

**Series:** Java Interview + Development Mastery Series **Book Number:** 02 of 24 **Java Versions**

**Covered:** Java 7/8 baseline OOP mechanics; Java 8 interface default/static methods referenced (full depth in Book 07); Java 17/21 sealed classes previewed as a forward pointer (full depth in Book 07)

**Prerequisites:** Book 01 — Java Fundamentals (classes, objects, constructors, `this`, `static` / `final`, access modifiers) **Next Book:** 03\_JVM\_Memory\_Management.md

### How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

**Telugu:** Book 01 లో మీరు class/object/constructor basics నేర్చుకున్నారు. ఈ పుస్తకం లో మనం **OOP యొక్క నాలుగు స్తంభాలు** (Encapsulation, Abstraction, Inheritance, Polymorphism) deep గా, SOLID principles తో సహా నేర్చుకుంటాము — ఇవి senior interview లో అత్యంత ఎక్కువగా cross-question చేయబడే topics.

**English:** Book 01 gave you class/object mechanics. This book goes deep into the **Four Pillars of OOP**, SOLID principles, and design vocabulary (coupling, cohesion, composition vs aggregation) — the single most cross-questioned area in Java interviews at every seniority level.

### Learning Objectives

1. Explain and apply Encapsulation, Abstraction, Inheritance, Polymorphism with real code.
2. Distinguish compile-time (overloading) vs runtime (overriding) polymorphism precisely.
3. Understand dynamic method dispatch internally (vtable-style resolution).
4. Master `super`, constructor chaining across a class hierarchy.
5. Know when to use abstract classes vs interfaces.
6. Understand IS-A vs HAS-A, and Composition vs Aggregation vs Association.
7. Apply all 5 SOLID principles with before/after code.
8. Explain coupling and cohesion, and design low-coupling/high-cohesion classes.
9. Answer senior-level OOP cross-questions confidently in Telugu and English.

### Table of Contents

| Ch. | Title                                |
|-----|--------------------------------------|
| 1   | The Four Pillars — Big Picture Recap |
| 2   | Encapsulation — Deep Dive            |
| 3   | Abstraction — Deep Dive              |
| 4   | Inheritance                          |

| Ch. | Title                                                                                |
|-----|--------------------------------------------------------------------------------------|
| 5   | super , Constructor Chaining Across Hierarchies                                      |
| 6   | Polymorphism — Compile-Time vs Runtime                                               |
| 7   | Method Overloading — Deep Dive                                                       |
| 8   | Method Overriding & Dynamic Dispatch — Deep Dive                                     |
| 9   | Abstract Classes — Deep Dive                                                         |
| 10  | Interfaces — Deep Dive                                                               |
| 11  | Abstract Class vs Interface — Decision Framework                                     |
| 12  | IS-A vs HAS-A: Composition, Aggregation, Association                                 |
| 13  | SOLID Principles                                                                     |
| 14  | Coupling & Cohesion + Mini Project                                                   |
| —   | Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist |

---

# CHAPTER 1 — The Four Pillars: Big Picture Recap

## Telugu Explanation

OOP (Object-Oriented Programming) నాలుగు స్తంభాల మీద నిలబడుతుంది:

1. **Encapsulation** — data ని hide చేసి, controlled access ఇవ్వడం.
2. **Abstraction** — "ఏమి చేస్తుంది" చూపించి, "ఎలా చేస్తుంది" అనేది hide చేయడం.
3. **Inheritance** — ఒక class మరో class యొక్క properties/behavior ని reuse చేయడం (IS-A relationship).
4. **Polymorphism** — ఒకే name, వేర్వేరు రూపాల్లో behave అవ్వడం (many forms).

## Professional English Explanation

Object-Oriented Programming rests on four pillars: **Encapsulation** (data hiding + controlled access), **Abstraction** (exposing "what" while hiding "how"), **Inheritance** (code reuse via an IS-A relationship), and **Polymorphism** (one interface, many implementations/behaviors). Book 01 already gave you the class/object/constructor mechanics; this book builds the design-level thinking on top of that foundation.

## Quick Map: Where Book 01 Ends and Book 02 Begins

| Book 01 (mechanics)   | Book 02 (design pillars)                 |
|-----------------------|------------------------------------------|
| -----                 | -----                                    |
| class, object, fields | Encapsulation (WHY hide fields)          |
| constructors, this    | super, constructor chaining (hierarchy)  |
| methods, overloading  | Overloading as compile-time polymorphism |
| static/final          | Abstract classes/interfaces, SOLID       |

## Real-World Example

Telugu: ఒక Vehicle hierarchy లో — Car, Bike classes Vehicle ని extend చేస్తాయి (Inheritance), start() method prompt చేసినప్పుడు actual vehicle type ఆధారంగా వేరే behavior వస్తుంది (Polymorphism), internal engine logic hide చేయబడి ఉంటుంది (Encapsulation + Abstraction). English: A Vehicle hierarchy (Car, Bike extending Vehicle) is the textbook example combining all four pillars at once — this chapter sets up the vocabulary; Chapters 2–8 build each pillar in depth with this same domain.

## Interview Answer

"OOP has four pillars: encapsulation (data hiding via access control), abstraction (hiding implementation complexity behind a simple interface), inheritance (reusing behavior through an IS-A hierarchy), and polymorphism (the same method call behaving differently based on the actual object type)."

## Cross Questions

- Q: Are abstraction and encapsulation the same thing? → A: No — abstraction is about *design* (hiding complexity, exposing intent), encapsulation is about *implementation* (hiding data via

access modifiers). They're related but distinct, detailed in Ch.2-3.

- Q: Can you have polymorphism without inheritance? → A: Runtime polymorphism (overriding) needs inheritance/interface implementation; compile-time polymorphism (overloading) does not.

## **Mastery Check**

Without notes, define all four pillars in one sentence each, in both Telugu and English, and give one line of code demonstrating each.

---

## CHAPTER 2 — Encapsulation, Deep Dive

### Telugu Explanation

Encapsulation అంటే data (fields) ని `private` చేసి, controlled access (getters/setters/validated methods) ద్వారానే బయటి ప్రపంచం access చేసేలా design చేయడం. దీని వల్ల internal representation మార్చినా, external code break అవ్వదు, మరియు invalid state (e.g., negative balance) రాకుండా validate చేయచ్చు.

### Professional English Explanation

Encapsulation bundles data with the methods that operate on it, restricting direct external access to internal state. This enforces invariants (valid state), supports information hiding, and decouples a class's internal representation from its public contract — the internal implementation can change freely as long as the public API stays stable.

### Java Code

```
class BankAccount {
    private double balance;                // hidden state

    BankAccount(double initialBalance) {
        if (initialBalance < 0) throw new IllegalArgumentException("Initial balance cannot be negative");
        this.balance = initialBalance;
    }

    public double getBalance() { return balance; }    // controlled read

    public void deposit(double amount) {             // controlled, validated write
        if (amount <= 0) throw new IllegalArgumentException("Deposit must be positive");
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Withdrawal must be positive");
        if (amount > balance) throw new IllegalStateException("Insufficient funds");
        balance -= amount;
    }
}

public class EncapsulationDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(1000.0);
        acc.deposit(500.0);
        acc.withdraw(200.0);
        System.out.println("Balance: " + acc.getBalance());

        try {
            acc.withdraw(999999.0);
        } catch (IllegalStateException e) {
            System.out.println("Blocked invalid withdrawal: " + e.getMessage());
        }
    }
}
```

## Output

```
Balance: 1300.0
Blocked invalid withdrawal: Insufficient funds
```

## Internal Working

- Without encapsulation, `acc.balance = -99999`; would compile and silently corrupt state — no validation point exists.
- With `private balance` + validated methods, **every** state change funnels through one code path ( `deposit / withdraw` ), which is exactly where invariants (never negative, never over-withdrawn) are enforced.
- This is the mechanical foundation `private` fields provide (Book 01, Ch.10) — encapsulation is *why* that access-control mechanism exists.

## Real-World Example

Telugu: Real banking/payment systems లో balance direct గా ఎప్పుడూ public field గా ఉండదు — ప్రతి change ఒక validated, audited method ద్వారానే జరుగుతుంది, compliance & correctness కోసం. English: In production financial systems, state mutation always goes through validated, often audited, methods — this is encapsulation applied at scale, and it's why "tell, don't ask" (calling `account.withdraw(x)` instead of reading and mutating balance externally) is a core OOP idiom.

## Interview Answer

"Encapsulation means hiding an object's internal state behind `private` fields and exposing only validated, controlled methods to read or modify that state — protecting invariants and decoupling internal representation from the public API."

## Deep Interview Answer

"Encapsulation isn't just 'make fields private, add getters/setters' — blindly generating getters/setters for every field re-exposes the same mutability problem through a different door. True encapsulation means designing methods around *behavior* ( `withdraw()` , `deposit()` ) rather than raw data access, so the class alone is responsible for maintaining its invariants — a principle sometimes called 'Tell, Don't Ask'."

## Cross Questions

- Q: Is generating a getter and setter for every private field still "good encapsulation"? → A: Technically it compiles and hides fields, but if every field has an unrestricted setter, you've just re-exposed full mutability — better encapsulation restricts *which* changes are allowed and validates them.
- Q: How does encapsulation support maintainability? → A: You can change internal representation (e.g., store balance in cents as `long` instead of `double` ) without breaking any external caller, as long as the public method signatures stay the same.

- Q: Does encapsulation prevent all bugs? → A: No — it only prevents *external* invalid-state bugs; internal logic errors inside the class itself are still possible and need testing (Book 15).

## Tricky Questions

- Q: If a class exposes a getter that returns a mutable object (e.g., `List` or `Date`) directly, is encapsulation broken? → A: Yes — this is a common leak; the caller can mutate the internal object through the returned reference. Fix: return a defensive copy or an unmodifiable view.
- Q: Can encapsulation exist without `private`? → A: In practice no in Java — encapsulation relies on access modifiers to enforce the boundary; without restriction, there's no hiding, just convention.

## Coding Exercise

**L1:** Convert a `Person` class with public `name` / `age` fields into a properly encapsulated version with validation (age can't be negative). **L2:** Fix a "leaky getter" bug: a class returning its internal `List` directly — make it return an unmodifiable view instead. **L3:** Design a `Temperature` class storing Celsius internally but exposing both `getCelsius()` and `getFahrenheit()` — demonstrating representation hiding. **L4 (Interview):** Explain why "getter/setter for every field" is not automatically good encapsulation. **L5 (Senior):** Refactor a `ShoppingCart` class that currently exposes its internal `items` array/list directly, to fully encapsulate mutation behind `addItem()` / `removeItem()` with validation. **L6 (Mastery):** Explain, from memory, the "Tell, Don't Ask" principle and connect it back to why `private` exists at all.

---

# CHAPTER 3 — Abstraction, Deep Dive

---

## Telugu Explanation

Abstraction అంటే complex implementation details ని hide చేసి, user కి కావాల్సిన **essential features మాత్రమే** చూపించడం. ఉదాహరణకి, `car.start()` call చేసినప్పుడు, driver కి engine internal combustion process తెలియాల్సిన అవసరం లేదు — అతను కేవలం "start" అనే action మాత్రమే వాడతాడు.

## Professional English Explanation

Abstraction focuses on exposing *what* an object does (its contract/interface) while hiding *how* it does it (implementation details). In Java, abstraction is achieved primarily through **abstract classes** and **interfaces** — both let callers program against a contract without knowing the concrete implementation.

## Java Code

```
abstract class PaymentProcessor {
    abstract void processPayment(double amount);    // WHAT, not HOW

    void logTransaction(double amount) {            // shared concrete behavior
        System.out.println("Transaction logged: " + amount);
    }
}

class CreditCardProcessor extends PaymentProcessor {
    @Override
    void processPayment(double amount) {
        System.out.println("Processing $" + amount + " via Credit Card gateway...");
        logTransaction(amount);
    }
}

class UpiProcessor extends PaymentProcessor {
    @Override
    void processPayment(double amount) {
        System.out.println("Processing Rs." + amount + " via UPI gateway...");
        logTransaction(amount);
    }
}

public class AbstractionDemo {
    public static void main(String[] args) {
        PaymentProcessor processor = new UpiProcessor(); // caller only knows the abstraction
        processor.processPayment(499.0);

        processor = new CreditCardProcessor();
        processor.processPayment(1200.0);
    }
}
```

## Output

```
Processing Rs.499.0 via UPI gateway...
Transaction logged: 499.0
Processing $1200.0 via Credit Card gateway...
Transaction logged: 1200.0
```

## Internal Working

- The caller ( `main` ) only ever interacts through the `PaymentProcessor` **type** — it never needs to know whether the actual gateway logic involves HTTP calls, bank APIs, or SDKs.
- This is what enables adding a new processor (e.g., `NetBankingProcessor` ) later **without changing any caller code** — a direct real-world payoff of abstraction, and a preview of the Open/Closed Principle (Ch.13).

## Real-World Example

Telugu: Different payment gateways (Razorpay, Stripe, PayU) ని ఒకే `PaymentProcessor` abstraction వెనుక hide చేస్తే, business logic గేట్‌వే మారినా మారదు. English: Abstracting payment gateways behind a common interface/abstract class is exactly how real checkout systems support multiple providers without scattering `if (gateway == "stripe")` branches through business logic.

## Interview Answer

"Abstraction means exposing only the essential behavior (the contract) while hiding implementation complexity. In Java it's implemented via abstract classes and interfaces, letting client code depend on a stable contract instead of concrete implementation details."

## Deep Interview Answer

"Abstraction and encapsulation solve different problems: encapsulation protects an object's *internal state* from misuse; abstraction protects the *caller* from needing to understand implementation complexity, enabling implementations to vary or be swapped entirely (dependency inversion, Ch.13) without touching calling code. Abstraction is a design-time concept realized in Java primarily through abstract classes and interfaces (Ch.9–10)."

## Cross Questions

- Q: Is abstraction only achieved through abstract classes/interfaces? → A: Those are the primary language-level tools in Java, but any well-designed class with a clean public API and hidden internals is practicing abstraction at a smaller scale.
- Q: Can a class be both abstract (in design intent) and fully concrete (no `abstract` keyword)? → A: Yes — a regular class with a clean, minimal public API and well-hidden internals still embodies the *principle* of abstraction, even without the `abstract` keyword.

## Tricky Questions

- Q: Does abstraction mean "hide all details," including from subclasses? → A: No — abstraction is about hiding details from *callers/clients*; subclasses of an abstract class often need visibility into `protected` members to fulfill the contract.

## Coding Exercise

**L1:** Create an abstract `Shape` class with abstract `area()`, implemented differently by `Circle` and `Rectangle`. **L2:** Add a concrete shared method `describe()` to `Shape` calling the abstract `area()` (template-method flavor, formalized in Book 18). **L3:** Add a third payment processor (`NetBankingProcessor`) to the demo without modifying `main()`'s logic shape. **L4 (Interview):** Explain, with an example, the precise difference between abstraction and encapsulation. **L5 (Senior):** Design an abstraction for a notification system (`Email`, `SMS`, `Push`) that a service layer can call without knowing the concrete channel. **L6 (Mastery):** Explain from memory why abstraction enables adding new implementations without modifying existing caller code.

---

# CHAPTER 4 — Inheritance

---

## Telugu Explanation

Inheritance అంటే ఒక class (subclass/child) మరో class (superclass/parent) యొక్క fields మరియు methods ని **reuse** చేయడం — ఇది **IS-A relationship** ని represent చేస్తుంది ( Car IS-A Vehicle ). Java **single inheritance** మాత్రమే support చేస్తుంది classes కోసం (ఒక class ఒక్క superclass మాత్రమే extend చేయచ్చు), కానీ interfaces multiple implement చేయచ్చు.

## Professional English Explanation

Inheritance lets a subclass acquire fields and methods from a superclass, modeling an IS-A relationship. Java supports single inheritance for classes (avoiding the "Diamond Problem") while allowing a class to implement multiple interfaces.

## Java Code

```
class Vehicle {
    protected String brand;
    protected int speed;

    Vehicle(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }

    void start() {
        System.out.println(brand + " vehicle starting...");
    }

    void displayInfo() {
        System.out.println(brand + " running at " + speed + " km/h");
    }
}

class Car extends Vehicle {           // Car IS-A Vehicle
    int numDoors;

    Car(String brand, int speed, int numDoors) {
        super(brand, speed);          // must call superclass constructor
        this.numDoors = numDoors;
    }

    void openTrunk() {
        System.out.println(brand + " trunk opened.");
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Car car = new Car("Toyota", 180, 4);
        car.start();                  // inherited from Vehicle
        car.displayInfo();            // inherited from Vehicle
        car.openTrunk();              // defined in Car

        Vehicle v = car;              // upcasting - always safe
        v.start();                    // works - Vehicle reference, Car object
        // v.openTrunk();              // compile error - Vehicle type doesn't know Car's methods
    }
}
```

## Output

```
Toyota vehicle starting...
Toyota running at 180 km/h
Toyota trunk opened.
Toyota vehicle starting...
```

## Internal Working / Memory View

- A `Car` object on the heap contains **both** the fields inherited from `Vehicle` ( `brand` , `speed` ) and its own ( `numDoors` ) — inheritance doesn't create a separate `Vehicle` object; it's one object with a combined layout.
- **Upcasting** ( `Vehicle v = car;` ) is always safe and implicit — a `Car` is a `Vehicle` . **Downcasting** ( `Car c = (Car) v;` ) requires an explicit cast and can throw `ClassCastException` at runtime if `v` doesn't actually reference a `Car` .
- Method resolution for inherited methods still uses dynamic dispatch (Ch.8) if overridden; non-overridden methods simply execute the superclass's code directly.

HEAP

Car object:

```
[ from Vehicle: brand="Toyota", speed=180 ]  
[ from Car:      numDoors=4                ]
```

## Real-World Example

Telugu: `Employee` base class నుండి `Manager` , `Developer` subclasses extend చేయడం — common fields ( `name` , `salary` ) reuse అవుతాయి, role-specific behavior ( `approveLeave()` ) కేవలం `Manager` కి మాత్రమే add అవుతుంది. English: Domain hierarchies ( `Employee` → `Manager` / `Developer` , or `Vehicle` → `Car` / `Bike` ) are the standard inheritance use case — shared fields/behavior live in the base class, specialized behavior lives in subclasses.

## Interview Answer

"Inheritance lets a subclass reuse a superclass's fields and methods, modeling an IS-A relationship. Java allows single inheritance of classes but multiple interface implementation, avoiding ambiguity from the classic diamond problem."

## Deep Interview Answer

"Java sidesteps the diamond problem for classes entirely by disallowing multiple class inheritance. For interfaces, Java 8's default methods reintroduced a limited diamond scenario, resolved by requiring the implementing class to explicitly override the conflicting method if two interfaces provide clashing default implementations (detailed in Book 07)."

## Cross Questions

- Q: Why does Java not support multiple inheritance for classes? → A: To avoid the diamond problem — ambiguity when two superclasses define the same method/field and a subclass inherits from both.
- Q: What's the difference between upcasting and downcasting? → A: Upcasting (subclass → superclass reference) is implicit and always safe; downcasting (superclass → subclass reference) requires an explicit cast and can fail at runtime.
- Q: Does a subclass inherit private members of its superclass? → A: Private members exist in the object's memory layout but are **not accessible** directly by name in the subclass — only via

inherited public/protected methods that touch them.

## Tricky Questions

- Q: If `Vehicle v = new Car(...)`, and `Car` overrides `displayInfo()`, which version runs? → A: `Car`'s version — runtime polymorphism (dynamic dispatch) resolves based on the actual object type, not the reference type (fully explained in Ch.8).
- Q: Can a subclass constructor skip calling `super(...)`? → A: If you don't write it explicitly, the compiler inserts an implicit `super()` (no-arg) call as the first statement — which fails to compile if the superclass has no no-arg constructor available.

## Coding Exercise

**L1:** Create a 2-level hierarchy: `Animal` → `Dog`, with `Dog` adding a `fetch()` method. **L2:** Demonstrate upcasting and a correctly-guarded downcasting (using `instanceof` check first) between `Animal` and `Dog`. **L3:** Build a 3-level hierarchy (`Employee` → `Manager` → `SeniorManager`), each adding one field/method, and print info showing all inherited state. **L4 (Interview):** Why doesn't Java support multiple class inheritance, and how do interfaces provide a safer alternative for multiple "capabilities"? **L5 (Senior):** Identify when a deep inheritance hierarchy (4+ levels) becomes a design smell, and propose a composition-based alternative (bridges to Ch.12). **L6 (Mastery):** Explain, without notes, the heap memory layout of a `Car` object and how upcasting/downcasting affect what's *accessible*, not what *exists*.

---

## CHAPTER 5 — `super`, Constructor Chaining Across Hierarchies

---

### Telugu Explanation

`super` keyword parent class ని refer చేయడానికి వాడతారు — parent constructor call చేయడానికి ( `super(...)` ), parent method call చేయడానికి ( `super.methodName()` ), లేదా parent field access చేయడానికి ( `super.fieldName` , రీడ్ గా వాడతారు). ప్రతి subclass constructor యొక్క **మొదటి statement** గా implicit గా లేదా explicit గా `super(...)` call జరుగుతుంది.

### Professional English Explanation

`super` refers to the immediate superclass. It's used to invoke the superclass constructor ( `super(...)` , must be the first statement), call an overridden superclass method explicitly ( `super.methodName()` ), or rarely access a shadowed field. Constructor chaining across a hierarchy always runs superclass constructors **before** subclass constructor bodies — bottom of the hierarchy is initialized last, top-most ancestor first.

## Java Code

```
class Animal {
    Animal() {
        System.out.println("Animal constructor");
    }
    void makeSound() {
        System.out.println("Some generic animal sound");
    }
}

class Dog extends Animal {
    Dog() {
        super(); // explicit, though implicit if omitted
        System.out.println("Dog constructor");
    }

    @Override
    void makeSound() {
        super.makeSound(); // call parent's version explicitly
        System.out.println("Woof! Woof!");
    }
}

class Puppy extends Dog {
    Puppy() {
        System.out.println("Puppy constructor");
    }
}

public class SuperDemo {
    public static void main(String[] args) {
        new Puppy();
        System.out.println("---");
        new Dog().makeSound();
    }
}
```

## Output

```
Animal constructor
Dog constructor
Puppy constructor
---
Animal constructor
Dog constructor
Some generic animal sound
Woof! Woof!
```

## Internal Working

- Constructor execution order is always **top-down**: `Animal()` → `Dog()` → `Puppy()`, regardless of which class you instantiate — because each subclass constructor's first act (explicit or compiler-inserted) is calling its immediate superclass constructor.

- `super.makeSound()` inside `Dog.makeSound()` explicitly invokes the **overridden** parent implementation — without it, only `Dog` 's version would run (this is how you "extend" rather than fully "replace" inherited behavior).

```
Instantiating Puppy:
Puppy() called
-> implicit super() -> Dog() called
    -> explicit super() -> Animal() called
        [Animal's body runs first]
    [Dog's body runs next]
[Puppy's body runs last]
```

## Real-World Example

Telugu: `AuditableEntity` base class లో `createdAt` set చేసే logic ఉంటే, subclasses ( `Order` , `Invoice` ) `super(...)` ద్వారా ఆ initialization ని guarantee గా inherit చేస్తాయి. English: Base classes providing common initialization (audit fields, IDs) rely on constructor chaining via `super(...)` to guarantee that shared setup always runs — a pattern used constantly in JPA entity base classes (Book 13).

## Interview Answer

"`super` refers to the immediate superclass — used to call its constructor ( `super(...)` , always first statement), or explicitly invoke its overridden method ( `super.method()` ). Constructor chaining always executes top-down through the hierarchy: the topmost ancestor's constructor runs first."

## Cross Questions

- Q: Can `super(...)` and `this(...)` both appear in the same constructor? → A: No — both must be the first statement, so only one can be used per constructor.
- Q: What happens if the superclass has no no-arg constructor and the subclass doesn't explicitly call `super(args)` ? → A: Compile error — the compiler's implicit `super()` call fails because no matching no-arg constructor exists.
- Q: Why call `super.method()` instead of just re-implementing the whole logic in the subclass? → A: To reuse and extend existing behavior (DRY) rather than duplicating it — common in logging, validation, or template-style extension.

## Tricky Questions

- Q: If `Animal` 's constructor calls an overridable method that `Dog` overrides, which version runs during `Animal` 's constructor execution? → A: `Dog` 's overridden version runs — because dynamic dispatch (Ch.8) is based on the actual object type even during superclass construction. This is a well-known Java gotcha: calling overridable methods from a constructor can operate on a not-yet-fully-initialized subclass state, and is generally discouraged.

## Coding Exercise

**L1:** Build a 3-level hierarchy and print the constructor execution order (like the demo) with your own domain (e.g., `Shape` → `Polygon` → `Triangle`). **L2:** Use `super.method()` to extend (not replace) a parent's `toString()` in a child class. **L3:** Reproduce the "overridable method called from

constructor" gotcha from the Tricky Question above and explain the surprising output. **L4**

**(Interview):** Explain why `super(...)` / `this(...)` must be the first statement in a constructor. **L5**

**(Senior):** Design a base `AuditableEntity` class with `createdAt` / `updatedAt` fields set via constructor, and two subclasses chaining through it correctly. **L6 (Mastery):** Recite, from memory, the exact constructor execution order for a 4-level hierarchy without running code.

---

# CHAPTER 6 — Polymorphism: Compile-Time vs Runtime

## Telugu Explanation

Polymorphism అంటే "అనేక రూపాలు" (many forms) — ఒకే method name వేర్వేరు సందర్భాల్లో వేర్వేరు గా behave అవ్వడం. రెండు రకాలు: **Compile-time polymorphism** (Method Overloading — ఏ method call అవ్వాలో compiler decide చేస్తుంది, argument types ఆధారంగా) మరియు **Runtime polymorphism** (Method Overriding — ఏ version run అవ్వాలో JVM runtime లో, actual object type ఆధారంగా decide చేస్తుంది).

## Professional English Explanation

Polymorphism means "many forms" for the same operation. **Compile-time (static) polymorphism** is achieved via method overloading — the compiler resolves which overload to call based on argument types at compile time. **Runtime (dynamic) polymorphism** is achieved via method overriding — the JVM resolves which implementation to execute based on the object's actual runtime type, not the reference's declared type.

## Java Code — Both Forms Side by Side

```
class Calculator {
    int add(int a, int b) { return a + b; }           // overload 1
    double add(double a, double b) { return a + b; } // overload 2 - compile-time choice
}

class Shape {
    double area() { return 0.0; }
}

class Circle extends Shape {
    double radius;
    Circle(double radius) { this.radius = radius; }
    @Override double area() { return Math.PI * radius * radius; } // runtime choice
}

class Square extends Shape {
    double side;
    Square(double side) { this.side = side; }
    @Override double area() { return side * side; }
}

public class PolymorphismDemo {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(2, 3));           // compiler picks int overload
        System.out.println(calc.add(2.5, 3.5));       // compiler picks double overload

        Shape[] shapes = { new Circle(2), new Square(3) };
        for (Shape s : shapes) {
            System.out.println(s.area());             // JVM picks based on actual object at
runtime
        }
    }
}
```

## Output

```
5
6.0
12.566370614359172
9.0
```

## Internal Working

- **Compile-time polymorphism:** the compiler inspects argument types at the call site and binds the call to a specific method **before the program ever runs** — this is why it's also called "static binding" or "early binding".
- **Runtime polymorphism:** the compiler only checks that `area()` exists on the declared type `Shape`; the **actual method invoked** is determined at runtime by looking at the object's real class — this is "dynamic binding" or "late binding", implemented internally via a method-lookup table conceptually similar to a "vtable" (detailed further in Ch.8).

## Comparison Table

| Aspect                  | Compile-Time (Overloading)               | Runtime (Overriding)                                           |
|-------------------------|------------------------------------------|----------------------------------------------------------------|
| Resolved by             | Compiler, using argument types           | JVM, using actual object type                                  |
| Also called             | Static / early binding                   | Dynamic / late binding                                         |
| Requires inheritance?   | No                                       | Yes (or interface implementation)                              |
| Return type flexibility | Can differ freely                        | Must be same or covariant                                      |
| Performance             | Slightly faster (resolved ahead of time) | Tiny dispatch overhead (negligible in practice, JIT-optimized) |

## Real-World Example

Telugu: `List<Shape>` లో వేర్వేరు shapes ఉంచి, ప్రతి దానికీ `area()` call చేస్తే, ఒక్క loop code తోనే వేర్వేరు shapes కి సరైన calculation జరుగుతుంది — ఇదే Polymorphism యొక్క production value: **code reuse without type-checking branches**. English: This is exactly why polymorphism matters in production: a single loop over a `List<Shape>` correctly computes the right area for every shape type, with zero `if (shape instanceof Circle)` branching — new shape types can be added without touching this loop at all (Open/Closed Principle, Ch.13).

## Interview Answer

"Compile-time polymorphism is method overloading, resolved by the compiler based on argument types before runtime. Runtime polymorphism is method overriding, resolved by the JVM based on the object's actual type at the moment of the call — this is what enables writing code against an abstraction that correctly behaves differently per concrete implementation."

## Cross Questions

- Q: Why is overloading called "static" and overriding called "dynamic"? → A: Overloading is bound at compile time (static structure of the code); overriding is bound at runtime based on the dynamic (actual) type of the object.
- Q: Can constructors be polymorphic (overridden)? → A: No — constructors aren't inherited, so overriding doesn't apply to them; only overloading applies to constructors.
- Q: Does polymorphism apply to fields? → A: No — field access is resolved at compile time based on the **reference type**, not the object's actual type (a classic trick question, detailed further in Ch.8).

## Tricky Questions

- Q: `Shape s = new Circle(2); System.out.println(s.radius);` — valid? → A: Compile error — `radius` isn't declared on `Shape`; fields are resolved by declared (reference) type, unlike overridden methods.
- Q: If `Square` had a *field* named `area` (not method) shadowing `Shape`'s field of the same name, which one would `((Shape) square).area` access? → A: `Shape`'s field — field access is never polymorphic; it's resolved statically by the reference type.

## Coding Exercise

**L1:** Write 3 overloaded `print()` methods (`int`, `String`, `double`) and call each. **L2:** Build a `Shape / Circle / Square / Triangle` hierarchy and compute total area of a `Shape[]` array in one loop. **L3:** Demonstrate the field-shadowing trick question above with your own hierarchy. **L4 (Interview):** Explain, precisely, why overriding is called "runtime polymorphism" using the vtable/dynamic-dispatch concept. **L5 (Senior):** Refactor a method full of `if (obj instanceof TypeA) ... else if (obj instanceof TypeB) ...` branches into clean runtime polymorphism. **L6 (Mastery):** Explain both forms of polymorphism from memory using the comparison table, without looking.

---

# CHAPTER 7 — Method Overloading, Deep Dive

## Telugu Explanation

Method Overloading అంటే ఒకే class లో, ఒకే method name తో, వేర్వేరు **parameter list** (number, type, order) ఉన్న multiple methods define చేయడం. Return type మాత్రమే వేరుగా ఉంటే overloading కాదు — compile error వస్తుంది.

## Professional English Explanation

Overloading means defining multiple methods with the same name but different parameter lists (differing in number, type, or order of parameters) within the same class (or between a class and subclass). It is resolved entirely at compile time based on the static types of arguments at the call site.

## Java Code — Overload Resolution Order

```
class OverloadResolution {
    void show(int x) { System.out.println("int version: " + x); }
    void show(long x) { System.out.println("long version: " + x); }
    void show(double x) { System.out.println("double version: " + x); }
    void show(Object x) { System.out.println("Object version: " + x); }
    void show(int... x) { System.out.println("varargs version: " + x.length + " args"); }
}

public class OverloadingDemo {
    public static void main(String[] args) {
        OverloadResolution r = new OverloadResolution();
        r.show(5);           // exact match -> int version
        r.show(5L);          // exact match -> long version
        r.show(5.0);         // exact match -> double version
        r.show("hello");     // no numeric match -> Object version
        r.show(1, 2, 3);     // no exact/widening match -> varargs version
        byte b = 2;
        r.show(b);           // no exact byte match -> widens to int version (closest widening)
    }
}
```

## Output

```
int version: 5
long version: 5
double version: 5.0
Object version: hello
varargs version: 3 args
int version: 2
```

## Internal Working — Java's Overload Resolution Priority

1. **Exact match** (no conversion needed).
2. **Widening primitive conversion** ( byte → short → int → long → float → double ).

3. **Autoboxing/unboxing** (Book 01, Ch.12) — tried only if no widening match exists.

4. **Varargs** — tried last, only if nothing else matches.

This ordering is why `r.show(b)` (a `byte`) picks the `int` overload via widening, rather than boxing to `Byte` and matching `Object`, or matching varargs — widening is tried before boxing and before varargs.

## Real-World Example

Telugu: JDK లో `String.valueOf(int)`, `String.valueOf(double)`, `String.valueOf(Object)` వంటి overloads real-world example — ఒకే API name, వేర్వేరు input types కి సరైన conversion. English: `String.valueOf(...)`, `Arrays.asList(...)`, and countless JDK utility methods are heavily overloaded — the same design idiom used in this chapter's demo.

## Interview Answer

"Overloading means multiple methods sharing a name but differing in parameter list, resolved entirely at compile time. Java resolves ambiguity using a strict priority: exact match, then widening, then autoboxing, then varargs — in that order."

## Deep Interview Answer

"Overload resolution never considers return type — two methods differing only in return type are a compile error, not valid overloads, because the compiler resolves calls purely from the call site's argument types, without knowing (or caring) what the caller does with the return value. This is fundamentally different from overriding, where the return type participates via covariant return type rules (Ch.8)."

## Cross Questions

- Q: Can overloaded methods differ only by throwing different exceptions? → A: No — exceptions thrown aren't part of the method signature used for overload resolution.
- Q: Is overloading possible across a parent-child relationship? → A: Yes — a subclass can define a method with the same name but different parameters than a superclass method; that's overloading (not overriding, since the signature differs).
- Q: What happens with ambiguous overloads, e.g., `show(null)` when both `show(String)` and `show(Object)` exist? → A: The compiler picks the **most specific** applicable type — `show(String)` wins here since `String` is more specific than `Object`. If two unrelated overloads are equally specific, it's a compile-time ambiguity error.

## Tricky Questions

- Q: Given `void f(int x)` and `void f(Integer x)`, what does `f(5)` call? → A: `f(int)` — exact primitive match beats autoboxing every time.
- Q: Given only `void f(long x)` and `void f(Integer x)`, what does `f(5)` (an `int` literal) call? → A: `f(long)` — widening (`int`→`long`) is tried before autoboxing (`int`→`Integer`).

## Coding Exercise

**L1:** Write overloaded `max()` methods for `int`, `double`, and 3 `int` s. **L2:** Reproduce and explain the `byte` widening-to-`int` example from the chapter with your own overload set. **L3:** Create an ambiguous-overload scenario deliberately (two equally-specific applicable overloads) and observe the compiler error message. **L4 (Interview):** State Java's exact overload resolution priority order from memory. **L5 (Senior):** Design an overloaded `Logger.log(...)` API (`String`, `String+Throwable`, `String+Object...`) that avoids ambiguity for common call patterns. **L6 (Mastery):** Predict, without running code, which overload is chosen for 5 different call expressions mixing primitives, wrappers, and varargs.

---

# CHAPTER 8 — Method Overriding & Dynamic Dispatch, Deep Dive

---

## Telugu Explanation

Method Overriding అంటే subclass, తన superclass లో ఉన్న method ని **అదే signature** తో తిరిగి define చేయడం, వేరే implementation ఇవ్వడానికి. Runtime లో, JVM ఏ method run చేయాలో object యొక్క **actual type** చూసి decide చేస్తుంది — దీన్నే **Dynamic Method Dispatch** అంటారు.

## Professional English Explanation

Overriding means a subclass redefines a superclass's instance method with the same signature (same name, parameters, and a compatible/covariant return type) to provide specialized behavior. At runtime, the JVM uses **dynamic method dispatch** — it looks up the method in the actual (runtime) class of the object, not the reference's declared (compile-time) type.

## Overriding Rules

1. Same method name, same parameter list.
2. Return type must be the same, or a **covariant** subtype (since Java 5).
3. Access modifier can't be more restrictive than the superclass method (can widen, e.g., `protected` → `public`, but not narrow).
4. Cannot throw new/broader **checked** exceptions than the overridden method (Book 04).
5. `static`, `private`, and `final` methods **cannot** be overridden (static can be hidden, private isn't inherited, final is explicitly locked).

## Java Code

```
class Animal {
    protected Animal reproduce() {                // return type: Animal
        System.out.println("Animal reproducing");
        return new Animal();
    }
    void makeSound() { System.out.println("Some sound"); }
}

class Dog extends Animal {
    @Override
    public Dog reproduce() {                        // covariant return type: Dog (subtype of
Animal)
        System.out.println("Dog reproducing");
        return new Dog();
    }
    @Override
    void makeSound() { System.out.println("Bark!"); }
}

public class OverridingDemo {
    public static void main(String[] args) {
        Animal a = new Dog();                      // reference type: Animal, actual type: Dog
        a.makeSound();                              // Dog's version runs - dynamic dispatch
        Animal baby = a.reproduce();                  // Dog's version runs, returns a Dog (covariant)

        System.out.println(baby instanceof Dog);
    }
}
```

## Output

```
Bark!
Dog reproducing
true
```

## Internal Working — Dynamic Dispatch (vtable concept)

- Every class effectively has a conceptual **method table** mapping method signatures to the actual bytecode to execute. When a subclass overrides a method, its entry **replaces** the superclass's entry in that lookup for objects of that subclass type.
- At the call site `a.makeSound()`, the compiler only checks that `Animal` **declares** `makeSound()` (compile-time type checking). At runtime, the JVM looks at `a`'s actual object header (which records its real class, `Dog`) and dispatches to `Dog`'s method table entry.
- This lookup happens on every polymorphic call, but the JIT compiler (Book 03) heavily optimizes repeated dispatch to the same concrete type, so the runtime cost in practice is negligible for typical code.

```
Reference a (declared type: Animal) ---> Dog object (actual type: Dog)
|
[Dog's method table]
makeSound -> Dog.makeSound()
reproduce -> Dog.reproduce()
```

## Real-World Example

Telugu: NotificationService interface కి EmailNotification, SmsNotification implementations ఉంటే, service.send() call చేసినప్పుడు actual object type ఆధారంగా సరైన channel కి message వెళ్తుంది — ఇదే dynamic dispatch production లో. English: Every strategy-style service layer ( PaymentGateway , NotificationChannel , DiscountPolicy ) relies on dynamic dispatch — calling one method name and getting the behavior appropriate to the concrete implementation actually wired in (often via Spring dependency injection, Book 11).

## Interview Answer

"Overriding lets a subclass provide a specialized implementation of a superclass's method with the same signature. The JVM resolves which version to run at runtime using dynamic method dispatch, based on the object's actual class — not the reference's declared type — which is the mechanism underlying runtime polymorphism."

## Deep Interview Answer

"Dynamic dispatch is conceptually implemented via a per-class method table (often analogized to C++'s vtable, though the JVM's actual mechanism — invokevirtual bytecode plus inline caching and JIT devirtualization — is more sophisticated). Each object's header references its actual class's method table; overriding replaces entries in that table for the subclass. static , private , and final methods bypass this entirely — they're resolved at compile time ( invokestatic / invokespecial ), which is exactly why they can't be overridden."

## Cross Questions

- Q: Can you override a method to throw a broader checked exception than the parent? → A: No — this would violate the Liskov Substitution Principle (Ch.13) since callers relying on the parent's checked-exception contract could be broken; unchecked exceptions have no such restriction.
- Q: Can you override a public method as protected ? → A: No — overriding can only widen or keep the same access level, never narrow it.
- Q: What is "method hiding," and how does it differ from overriding? → A: Hiding applies to static methods redeclared in a subclass with the same signature — resolution is based on the reference type at compile time, not the object's actual type, unlike true overriding.

## Tricky Questions

- Q: Animal a = new Dog(); a.makeSound(); vs a hypothetical static hiddenMethod() on both classes called the same way — do both dispatch to Dog 's version? → A: No — the instance method dispatches to Dog (dynamic dispatch), but a static method call resolves based on the

**compile-time reference type** ( `Animal` ), so it would call `Animal` 's static method — a very common trick question.

- Q: Can a subclass overriding method have a *narrower* checked exception list, or none at all? → A: Yes — narrowing or removing checked exceptions is always allowed; only widening/adding new checked exceptions is disallowed.

## Coding Exercise

**L1:** Build an `Animal / Dog / Cat` hierarchy overriding `makeSound()`, and call it through an `Animal[]` array. **L2:** Demonstrate covariant return types by overriding a method that returns the superclass type with a subclass return type. **L3:** Demonstrate the "static method hiding vs instance method overriding" distinction from the Tricky Question with runnable code. **L4 (Interview):** List all 5 overriding rules from memory and explain why each exists. **L5 (Senior):** Review a hierarchy where a subclass has *widened* a checked exception in an overridden method — explain why this fails to compile and how it protects callers (LSP tie-in). **L6 (Mastery):** Explain dynamic dispatch using the method-table mental model, from memory, including why `static / private / final` methods are excluded from it.

---

## CHAPTER 9 — Abstract Classes, Deep Dive

---

### Telugu Explanation

Abstract class అంటే **directly instantiate చేయలేని** class, ఇందులో abstract methods (body లేని, subclass లో తప్పనిసరిగా implement చేయాల్సినవి) మరియు concrete methods (already implemented, shared behavior) రెండూ ఉండొచ్చు.

### Professional English Explanation

An abstract class cannot be instantiated directly; it can declare abstract methods (no body — subclasses **must** implement them) alongside fully implemented concrete methods (shared, reusable behavior). It's ideal when related classes share both common state/behavior **and** a contract of methods that must vary per subclass.

## Java Code

```
abstract class Employee {
    protected String name;
    protected double baseSalary;

    Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    abstract double calculateBonus();           // MUST be implemented by subclasses

    double calculateTotalPay() {                // concrete, shared logic reusing the abstract
method      return baseSalary + calculateBonus();
    }

    void printPaySlip() {
        System.out.println(name + " total pay: " + calculateTotalPay());
    }
}

class Developer extends Employee {
    Developer(String name, double baseSalary) { super(name, baseSalary); }

    @Override
    double calculateBonus() { return baseSalary * 0.10; }
}

class Manager extends Employee {
    Manager(String name, double baseSalary) { super(name, baseSalary); }

    @Override
    double calculateBonus() { return baseSalary * 0.20; }
}

public class AbstractClassDemo {
    public static void main(String[] args) {
        Employee[] employees = { new Developer("Anitha", 60000), new Manager("Kiran", 90000) };
        for (Employee e : employees) e.printPaySlip();

        // Employee e = new Employee("X", 1000); // compile error - cannot instantiate abstract
class
    }
}
```

## Output

```
Anitha total pay: 66000.0
Kiran total pay: 108000.0
```

## Internal Working

- `calculateTotalPay()` is a **Template Method**-flavored pattern (formalized fully in Book 18): the abstract class defines the *algorithm skeleton* ( `baseSalary + calculateBonus()` ), delegating one

variable step ( `calculateBonus()` ) to subclasses.

- Because `Employee` has an abstract method, the compiler forbids `new Employee(...)` entirely — an abstract class exists purely to be extended, never instantiated on its own.
- An abstract class **can** have constructors (called via `super(...)` from subclasses) even though it can never be instantiated directly — its constructor only runs as part of building a concrete subclass instance.

## Real-World Example

Telugu: Real payroll systems లో `Employee` base class లో common logic ( `calculateTotalPay` ) పెట్టి, bonus calculation matter role-specific ను override చేస్తారు — code duplication తగ్గుతుంది. English: This exact "shared algorithm + one varying step" shape appears constantly in production — payroll, discount calculation, report generation — anywhere multiple subclasses share most of an algorithm but customize one piece.

## Interview Answer

"An abstract class can't be instantiated and may mix abstract methods (no implementation, mandatory for subclasses) with concrete methods (shared implementation). It's the right tool when related classes share both state and some common behavior, but need to vary specific steps."

## Deep Interview Answer

"Abstract classes support instance state (fields), constructors, and access-modifier flexibility ( `protected` , `private` helper methods) that plain interfaces historically couldn't — although Java 8/9's default and private interface methods (Book 07) narrowed this gap somewhat. The key remaining distinction is state: an abstract class can hold and initialize instance fields via constructors; an interface cannot hold instance state at all."

## Cross Questions

- Q: Can an abstract class have a constructor? → A: Yes — it runs via `super(...)` when a concrete subclass is instantiated, even though the abstract class itself can never be `new` 'd directly.
- Q: Can an abstract class have zero abstract methods? → A: Yes — a class can be marked `abstract` purely to prevent direct instantiation, even with all methods concrete (a rare but valid design choice).
- Q: Must every subclass implement every abstract method? → A: Yes, unless the subclass is itself declared `abstract` , deferring the obligation further down the hierarchy.

## Tricky Questions

- Q: Can an abstract method be `private` or `final` ? → A: No — `private` methods can't be overridden (so declaring one abstract is contradictory), and `final` explicitly forbids overriding, which an abstract method requires.
- Q: Can an abstract class implement an interface without implementing all its methods? → A: Yes — an abstract class is allowed to leave some (or all) interface methods unimplemented, passing the obligation to its own subclasses.

## Coding Exercise

**L1:** Build an abstract `Shape` class with abstract `area()` / `perimeter()` and a concrete `describe()` using both. **L2:** Add a third `Employee` subclass ( `Intern` ) with its own bonus rule, without touching `Employee` or other subclasses. **L3:** Demonstrate an abstract class with zero abstract methods, explaining why you'd still mark it `abstract` . **L4 (Interview):** Explain why abstract classes can hold instance state but plain interfaces (pre-Java-8 style) could not. **L5 (Senior):** Design an abstract `ReportGenerator` class with a template method `generate()` calling abstract `fetchData()` / `formatOutput()` steps, implemented by `PdfReportGenerator` / `CsvReportGenerator` . **L6 (Mastery):** Explain, from memory, exactly when a class must be declared `abstract` (hint: any unimplemented abstract method forces it).

---

# CHAPTER 10 — Interfaces, Deep Dive

## Telugu Explanation

Interface అంటే పూర్తిగా (traditionally) abstract "contract" — ఏ class ఏమి చేయాలో define చేస్తుంది, ఎలా చేయాలో కాదు. ఒక class multiple interfaces implement చేయచ్చు — ఇదే Java లో "multiple inheritance of type" సాధించే మార్గం. (Java 8+ interfaces `default / static` methods కూడా కలిగి ఉండగలవు — పూర్తి వివరాలు Book 07 లో.)

## Professional English Explanation

An interface defines a contract — a set of method signatures a class agrees to implement — without dictating implementation. A class can implement multiple interfaces, which is how Java achieves multiple inheritance of *type* (though not of implementation, in the classic sense) while avoiding the diamond problem for state. Since Java 8, interfaces can also carry `default` and `static` methods with bodies (full coverage in Book 07); this chapter focuses on foundational interface mechanics.

## Java Code

```
interface Drivable {
    void drive();                // implicitly public abstract
    int MAX_SPEED = 200;        // implicitly public static final (a "constant")
}

interface Chargeable {
    void charge();
}

class ElectricCar implements Drivable, Chargeable { // multiple interface implementation
    @Override
    public void drive() {        // must be public - can't narrow
        System.out.println("Driving silently at up to " + MAX_SPEED + " km/h");
    }
    @Override
    public void charge() {
        System.out.println("Charging battery...");
    }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        ElectricCar car = new ElectricCar();
        car.drive();
        car.charge();

        Drivable d = car;        // program to the interface
        d.drive();
        // d.charge();            // compile error - Drivable type doesn't declare
    }
}
```

## Output

```
Driving silently at up to 200 km/h
Charging battery...
Driving silently at up to 200 km/h
```

## Internal Working

- Every interface field is implicitly `public static final` — interfaces cannot hold per-instance mutable state, only shared constants.
- Every (traditional) interface method is implicitly `public abstract` unless marked `default / static / private` (Java 8+, Book 07) — implementing classes must use `public` when overriding, since narrowing access is never allowed (Ch.8's overriding rules apply here too).
- Implementing multiple interfaces is safe from the diamond problem because interfaces (pre-Java-8) carried no implementation to conflict — only method *signatures*. Java 8's `default` methods reintroduce a narrow version of this conflict, resolved by mandatory explicit override (Book 07).

## Real-World Example

Telugu: `Comparable`, `Runnable`, `Serializable` వంటి JDK interfaces — ఒక class ఏకకాలంలో multiple capabilities కలిగి ఉండటానికి (sortable + runnable + serializable) వీటిని implement చేయచ్చు. English: JDK interfaces like `Comparable<T>`, `Runnable`, `Serializable`, and `AutoCloseable` are the standard way Java grants a class multiple independent "capabilities" — a class implementing all four gets sortability, threadability, serializability, and try-with-resources support simultaneously.

## Interview Answer

"An interface defines a contract of method signatures (and constants) without implementation, letting unrelated classes agree to the same capability. A class can implement multiple interfaces, which is how Java supports multiple inheritance of type while avoiding the diamond problem for implementation and state."

## Cross Questions

- Q: Can an interface have instance fields? → A: No — all interface fields are implicitly `public static final` constants, not per-instance mutable state.
- Q: Can an interface extend another interface? → A: Yes, and an interface can even extend multiple interfaces (unlike classes, which support only single class inheritance).
- Q: Can a class implement an interface without providing a method body, and still compile? → A: Only if the class itself is declared `abstract`, deferring the implementation obligation.

## Tricky Questions

- Q: If two interfaces both declare a method with the exact same signature and a class implements both, is there a conflict? → A: No conflict for traditional abstract methods — one

implementation in the class satisfies both interfaces simultaneously, since neither carries a body.

- Q: Can an interface be `final`? → A: No — a `final` interface could never be implemented, which is contradictory to what an interface is for; the compiler rejects this.

## Coding Exercise

**L1:** Create two interfaces ( `Flyable` , `Swimmable` ) and a `Duck` class implementing both. **L2:** Implement the JDK `Comparable<T>` interface on a custom `Student` class to sort by marks. **L3:** Demonstrate an interface constant ( `public static final` implicitly) being accessed both via the interface name and via an implementing class's instance. **L4 (Interview):** Explain exactly how Java achieves "multiple inheritance of type" safely using interfaces. **L5 (Senior):** Design a small plugin-style system where a `PaymentGateway` interface is implemented by 3 unrelated classes ( `Stripe` , `Razorpay` , `PayPal` stand-ins), and a service class depends only on the interface type. **L6 (Mastery):** Explain, from memory, why interface fields are always constants and why that's not a limitation for typical interface use cases.

---

# CHAPTER 11 — Abstract Class vs Interface: Decision Framework

## Telugu Explanation

"ఎప్పుడు abstract class వాడాలి, ఎప్పుడు interface వాడాలి?" — ఇది అత్యంత frequently అడిగే interview question. Rule of thumb: shared **state** + shared **partial implementation** అవసరమైతే abstract class; pure **capability contract**, multiple unrelated classes కి apply అవ్వాలంటే interface.

## Professional English Explanation

This is one of the most common OOP interview questions. The decision hinges on: does the type need to hold shared mutable state and provide substantial shared implementation (→ abstract class), or does it need to describe a capability/contract that unrelated classes across a hierarchy can adopt, possibly multiple at once (→ interface)?

## Decision Table

| Criterion                     | Abstract Class                                             | Interface                                                                           |
|-------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Multiple inheritance          | ✗ (single class extension only)                            | ✓ (implement many)                                                                  |
| Instance state (fields)       | ✓                                                          | ✗ (constants only)                                                                  |
| Constructors                  | ✓                                                          | ✗                                                                                   |
| Partial shared implementation | ✓ (concrete methods)                                       | ✓ since Java 8 ( default methods, Book 07) but limited to no state                  |
| "IS-A" strong relationship    | ✓ best fit                                                 | Weaker — more "CAN-DO"                                                              |
| Access modifiers on methods   | Full range ( private , protected , public )                | Historically public only; private allowed since Java 9 for internal helper methods  |
| Versioning existing types     | Adding a method breaks all subclasses unless abstract-safe | Adding an abstract method breaks implementers; default methods (Java 8+) avoid this |

## Java Code — Combining Both Correctly

```
interface Payable {                                // pure capability contract
    double calculatePay();
}

abstract class Person {                            // shared state + partial behavior
    protected String name;
    Person(String name) { this.name = name; }
    void greet() { System.out.println("Hello, I am " + name); }
}

class Contractor extends Person implements Payable { // IS-A Person, CAN-DO Payable
    private double hourlyRate;
    private int hours;

    Contractor(String name, double hourlyRate, int hours) {
        super(name);
        this.hourlyRate = hourlyRate;
        this.hours = hours;
    }

    @Override
    public double calculatePay() { return hourlyRate * hours; }
}

public class AbstractVsInterfaceDemo {
    public static void main(String[] args) {
        Contractor c = new Contractor("Meena", 500, 40);
        c.greet();
        System.out.println("Pay: " + c.calculatePay()); // from abstract class // from interface contract
    }
}
```

## Output

```
Hello, I am Meena
Pay: 20000.0
```

## Real-World Example

Telugu: `Person` (abstract class, shared identity/state) + `Payable` (interface, "can be paid" capability) కలిపి `Contractor` design చేయడం — real domain modeling లో ఇదే combined approach చాలా common. English: This "extend one abstract base for shared IS-A state, implement one-or-more interfaces for CAN-DO capabilities" pattern is exactly how real domain models are typically composed — a `Contractor` IS-A `Person` and CAN-BE `Payable`, `Taxable`, `Auditable`, etc.

## Interview Answer

"Use an abstract class when related types share state and substantial common implementation under a strong IS-A relationship. Use an interface when you're describing a capability/contract that possibly unrelated classes should adopt, especially when a class needs multiple such capabilities simultaneously."

## Cross Questions

- Q: Since Java 8 interfaces can have `default` methods with bodies, are abstract classes obsolete? → A: No — interfaces still can't hold instance state or constructors, which abstract classes provide; they solve different problems and are often combined, as shown above.
- Q: Can you extend an abstract class AND implement interfaces at the same time? → A: Yes, exactly as shown — `class X extends AbstractBase implements InterfaceA, InterfaceB`.
- Q: If you're not sure which to choose, what's a safe default? → A: Start with an interface (looser coupling, more flexible); introduce an abstract class only once you have genuine shared state/implementation to factor out.

## Tricky Questions

- Q: Can an abstract class implement an interface but leave some interface methods still abstract (unimplemented)? → A: Yes — this is completely legal; the concrete subclass further down must implement whatever remains.
- Q: Does adding a new method to an existing interface break existing implementers? → A: Only if it's a new **abstract** method — adding a `default` method (Java 8+) with a sensible fallback implementation doesn't break existing implementers, which is precisely why `default` methods were introduced (fully covered in Book 07).

## Coding Exercise

**L1:** Model `Bird` (abstract class: shared `name`, `eat()`) with `Flyable` / `Swimmable` interfaces, implemented differently by `Eagle`, `Penguin`, `Duck`. **L2:** Take an existing single large interface and split it into two smaller ones (Interface Segregation preview, Ch.13) implemented selectively. **L3:** Build the `Person` / `Payable` / `Contractor` example further, adding a `FullTimeEmployee` that's `Payable` differently (fixed salary, not hourly). **L4 (Interview):** Give 2 concrete scenarios where you'd unambiguously choose abstract class over interface, and vice versa. **L5 (Senior):** Review a hypothetical God-interface with 15 unrelated methods forced on every implementer — propose a split using both abstract classes and smaller interfaces. **L6 (Mastery):** Recreate the full decision table from memory with one example scenario per row.

---

# CHAPTER 12 — IS-A vs HAS-A: Composition, Aggregation, Association

---

## Telugu Explanation

**IS-A** relationship అంటే Inheritance ( Car IS-A Vehicle ). **HAS-A** relationship అంటే ఒక class లో మరో class యొక్క reference ఉండటం (Composition/Aggregation/Association) — Inheritance కాదు. మూడు రకాల HAS-A:

- **Composition** — strong ownership; part object, whole లేకుండా బతకదు ( Engine Car లోపలే create/destroy అవుతుంది).
- **Aggregation** — weak ownership; part, whole లేకుండా కూడా బతకగలదు ( Department కి Employee లు ఉంటారు, కానీ department delete అయినా employee independently exist అవ్వచ్చు).
- **Association** — general "uses-a" relationship, ఏ ownership లేకుండా ( Student Teacher ని "knows", ఇద్దరూ independent గా exist).

## Professional English Explanation

**IS-A** is inheritance. **HAS-A** is expressed through object references as fields, and comes in three strengths:

- **Composition** — strong ownership; the contained object's lifecycle is bound to the container's (created and destroyed with it).
- **Aggregation** — weak ownership; the contained object can outlive the container and can exist independently, possibly shared across containers.
- **Association** — a general "uses" or "knows about" relationship with no ownership implication either way.

## Java Code

```
// COMPOSITION: Engine cannot exist meaningfully outside a Car; created inside Car
class Engine {
    void start() { System.out.println("Engine starting..."); }
}
class Car {
    private final Engine engine = new Engine();    // Car owns and creates its Engine
    void start() { engine.start(); System.out.println("Car ready to drive"); }
}

// AGGREGATION: Employee can exist independently of Department
class Employee2 {
    String name;
    Employee2(String name) { this.name = name; }
}
class Department {
    private String deptName;
    private java.util.List<Employee2> employees;    // Department doesn't create these
    Employees

    Department(String deptName, java.util.List<Employee2> employees) {
        this.deptName = deptName;
        this.employees = employees;                // supplied externally - weak ownership
    }

    void printTeam() {
        System.out.println(deptName + " team: ");
        for (Employee2 e : employees) System.out.println(" - " + e.name);
    }
}

// ASSOCIATION: Student and Teacher are independent, just related
class Teacher { String subject; Teacher(String subject) { this.subject = subject; } }
class Student {
    String name;
    Teacher teacher;    // Student "knows" a Teacher, neither owns the other
    Student(String name, Teacher teacher) { this.name = name; this.teacher = teacher; }
}

public class RelationshipsDemo {
    public static void main(String[] args) {
        Car car = new Car();
        car.start();

        Employee2 e1 = new Employee2("Ravi");
        Employee2 e2 = new Employee2("Sita");
        Department dept = new Department("Engineering", java.util.List.of(e1, e2));
        dept.printTeam();
        // e1, e2 still fully usable even if 'dept' goes away - independent lifecycle

        Teacher t = new Teacher("Mathematics");
        Student s = new Student("Anitha", t);
        System.out.println(s.name + " is taught by a " + s.teacher.subject + " teacher");
    }
}
```

## Output

```
Engine starting...
Car ready to drive
Engineering team:
- Ravi
- Sita
Anitha is taught by a Mathematics teacher
```

## Internal Working

- **Composition** is signaled in code by the container **creating** the contained object internally (often in its own constructor, as `Engine engine = new Engine();` above) — the contained object has no independent existence outside.
- **Aggregation** is signaled by the container **receiving** the contained object(s) from outside (constructor parameter, setter) — the caller controls the contained objects' actual lifecycle.
- **Association** is the loosest form — a simple reference field with no lifecycle implication in either direction; UML often draws it as a plain line, aggregation as a line with a hollow diamond, composition as a line with a filled diamond.

## Real-World Example

Telugu: JPA లో @OneToMany relationships (Book 13) design చేసేటప్పుడు composition/aggregation తేడా ముఖ్యం — Order cascade=ALL, orphanRemoval=true తో OrderItem లని composition గా treat చేస్తే, order delete అయితే items కూడా delete అవుతాయి; weak aggregation అయితే వేరుగా handle చేయాలి.

English: This exact distinction drives real cascade/lifecycle decisions in JPA relationships (Book 13) — composition maps to `cascade = CascadeType.ALL, orphanRemoval = true` (child dies with parent), aggregation maps to a relationship without cascading delete (child can outlive the parent record).

## Interview Answer

"IS-A is inheritance. HAS-A is a reference-based relationship with three strengths: composition (strong ownership, contained object's lifecycle is bound to the container), aggregation (weak ownership, contained object can exist independently), and association (a general 'uses' relationship with no ownership either way)."

## Cross Questions

- Q: How do you tell composition from aggregation in code, mechanically? → A: Look at where the contained object is created — inside the container's own constructor/methods (composition) versus passed in from outside (aggregation).
- Q: Is "favor composition over inheritance" the same as this chapter's "composition"? → A: Related but broader — that principle (Ch.13's Liskov/design guidance) means preferring HAS-A (of any strength) over IS-A to reduce tight coupling from deep inheritance hierarchies; this chapter's "Composition" specifically refers to the *strong-ownership* flavor of HAS-A.
- Q: Can a class have both IS-A and HAS-A relationships simultaneously? → A: Yes, very commonly — e.g., `Car extends Vehicle` (IS-A) `has an Engine` (composition HAS-A).

## Tricky Questions

- Q: If `Department` 's constructor instead did `this.employees = new ArrayList<>(List.of(new Employee2("X")))`; internally, would that still be aggregation? → A: No — if `Department` *creates* the `Employee2` objects itself internally rather than receiving them, that's shifted toward composition-like ownership for those specific instances, even though `Employee2` conceptually *could* exist independently — the code-level signal (who creates it) is what determines the classification in a given design.
- Q: Is "favor composition over inheritance" ever wrong advice? → A: It's a strong default, but a genuine, stable IS-A relationship (e.g., `Square` IS-A `Shape`) is still correctly modeled with inheritance — the advice guards against overusing inheritance for convenience/code-reuse when the relationship isn't truly IS-A.

## Coding Exercise

**L1:** Identify IS-A vs HAS-A for 5 real-world pairs (e.g., "Wallet has Money", "Square is a Shape") and justify each. **L2:** Convert an aggregation example ( `Department` / `Employee2` ) so employees are shared across two departments (proving independent lifecycle). **L3:** Build a `House` (composition: `Room` s created internally) vs `Library` (aggregation: `Book` s supplied externally, can be moved to another library) pair. **L4 (Interview):** Explain, with the code-level "who creates it" heuristic, how to distinguish composition from aggregation in an unfamiliar codebase. **L5 (Senior):** Given a JPA-style `Order` / `OrderItem` relationship, decide composition vs aggregation and justify the corresponding cascade/orphan-removal behavior (conceptually, ahead of Book 13). **L6 (Mastery):** Explain all three HAS-A strengths plus IS-A from memory with a fresh, non-book example for each.

---

# CHAPTER 13 — SOLID Principles

## Telugu Explanation

SOLID అనేది 5 object-oriented design principles యొక్క acronym, maintainable, extensible code రాయడానికి:

- **S** — Single Responsibility Principle (ఒక class కి ఒక్కటే change కారణం ఉండాలి)
- **O** — Open/Closed Principle (extension కి open, modification కి closed)
- **L** — Liskov Substitution Principle (subclass, parent ని replace చేయగలగాలి, behavior break కాకుండా)
- **I** — Interface Segregation Principle (పెద్ద, monolithic interfaces కంటే చిన్న, focused interfaces better)
- **D** — Dependency Inversion Principle (concrete classes మీద కాకుండా abstractions మీద depend అవ్వాలి)

## Professional English Explanation

SOLID is a set of five foundational OOP design principles for maintainable, extensible software:

**S**ingle Responsibility, **O**pen/Closed, **L**iskov Substitution, **I**nterface Segregation, **D**ependency Inversion. Each is illustrated below with a "bad design → SOLID fix" pair.

### S — Single Responsibility Principle (SRP)

```
// BAD: one class does persistence AND business logic AND reporting
class BadInvoice {
    void calculateTotal() { /* ... */ }
    void saveToDatabase() { /* ... */ }
    void printReport() { /* ... */ }
}

// GOOD: each class has exactly one reason to change
class Invoice {
    double calculateTotal() { return 100.0; /* ... */ }
}
class InvoiceRepository {
    void save(Invoice invoice) { System.out.println("Saved to DB"); }
}
class InvoiceReportPrinter {
    void print(Invoice invoice) { System.out.println("Total: " + invoice.calculateTotal()); }
}
```

Telugu: Invoice class business logic మాత్రమే handle చేస్తుంది; save/print వేరే classes కి పంపబడ్డాయి — DB technology మారినా Invoice మారదు.

## O — Open/Closed Principle (OCP)

```
// Adding a new discount type requires NO changes to existing classes - only a new one
interface DiscountPolicy { double apply(double price); }
class NoDiscount implements DiscountPolicy { public double apply(double price) { return price; } }
class FestiveDiscount implements DiscountPolicy { public double apply(double price) { return price * 0.9; } }
// New requirement -> add ClearanceDiscount, touching nothing above (this is the point of Ch.3/6's abstraction+polymorphism)
class ClearanceDiscount implements DiscountPolicy { public double apply(double price) { return price * 0.5; } }
```

Telugu: కొత్త discount type add చేయడానికి existing code మార్చాల్సిన అవసరం లేదు — ఇదే "open for extension, closed for modification".

## L — Liskov Substitution Principle (LSP)

```
// CLASSIC VIOLATION: Square extends Rectangle but breaks expected behavior
class Rectangle {
    protected int width, height;
    void setWidth(int w) { this.width = w; }
    void setHeight(int h) { this.height = h; }
    int area() { return width * height; }
}
class Square extends Rectangle {
    @Override void setWidth(int w) { width = w; height = w; } // silently changes height too!
    @Override void setHeight(int h) { width = h; height = h; } // silently changes width too!
}
// A caller expecting Rectangle semantics (setWidth then setHeight independently) gets WRONG results with a Square
```

Telugu: Square, Rectangle ని extend చేసినా, setWidth() / setHeight() independent గా పనిచేయాలన్న parent contract violate చేస్తుంది — LSP violation. Fix: Square, Rectangle రెండూ ఒక common Shape abstraction నుండి వేరుగా extend కావాలి, IS-A relationship నిజంగా correct అయినప్పుడే వాడాలి (Ch.4/12 తో connect).

## I — Interface Segregation Principle (ISP)

```
// BAD: fat interface forces unrelated implementations to provide no-op/unsupported methods
interface WorkerBad { void work(); void eat(); }
class RobotBad implements WorkerBad {
    public void work() { System.out.println("Robot working"); }
    public void eat() { throw new UnsupportedOperationException("Robots don't eat!"); } // forced, wrong
}

// GOOD: split into focused interfaces
interface Workable { void work(); }
interface Eatable { void eat(); }
class Robot implements Workable { public void work() { System.out.println("Robot working"); } }
class Human implements Workable, Eatable {
    public void work() { System.out.println("Human working"); }
    public void eat() { System.out.println("Human eating"); }
}
```

Telugu: Robot కి eat() అవసరం లేదు — fat interface force చేస్తే wrong/broken implementation వస్తుంది; చిన్న, focused interfaces better.

## D — Dependency Inversion Principle (DIP)

```
// BAD: high-level class directly depends on a concrete low-level class
class MySQLDatabase { void save(String data) { System.out.println("Saved to MySQL: " + data); }
}
class BadOrderService {
    private MySQLDatabase db = new MySQLDatabase();    // tightly coupled to one concrete
implementation
    void placeOrder(String data) { db.save(data); }
}

// GOOD: depend on an abstraction; concrete implementation is injected
interface Database { void save(String data); }
class MySQLDb implements Database { public void save(String data) { System.out.println("Saved
to MySQL: " + data); } }
class MongoDB implements Database { public void save(String data) { System.out.println("Saved
to MongoDB: " + data); } }

class OrderService {
    private final Database db;
    OrderService(Database db) { this.db = db; }        // dependency injected, not hardcoded
    void placeOrder(String data) { db.save(data); }
}
```

Telugu: OrderService, ఏ database concrete class మీదనో కాకుండా, Database abstraction మీద depend అవుతుంది — database మారినా (MySQL నుండి MongoDB కి) OrderService మారదు. ఇదే Spring's Dependency Injection (Book 11) యొక్క foundational principle.

## Real-World Example

Telugu: Spring Framework మొత్తం Dependency Inversion Principle మీదనే build అయ్యింది — @Autowired ద్వారా abstractions (interfaces) inject చేయబడతాయి, concrete implementations కాదు (Book 11లో లోతుగా). English: DIP is literally the theoretical foundation of Spring's Dependency Injection (Book 11) — this chapter's OrderService / Database example is the exact shape of every @Service / @Repository pairing you'll write in Spring Boot (Book 12).

## Interview Answer

"SOLID is five principles for maintainable OOP design: Single Responsibility (one reason to change per class), Open/Closed (extend without modifying existing code), Liskov Substitution (subtypes must be substitutable for their base type without breaking correctness), Interface Segregation (prefer small, focused interfaces over large ones), and Dependency Inversion (depend on abstractions, not concrete implementations)."

## Cross Questions

- Q: How does OCP relate to polymorphism (Ch.6)? → A: OCP is achieved *through* polymorphism and abstraction — new behavior is added via new classes implementing an existing interface/abstract class, not by modifying existing conditional logic.

- Q: Give a concrete symptom of an LSP violation. → A: A subclass overriding a method to throw `UnsupportedOperationException`, or silently changing behavior in a way that breaks code written against the parent's contract (like the `Square / Rectangle` example).
- Q: How does DIP differ from simple abstraction (Ch.3)? → A: DIP specifically flips the *dependency direction* — high-level modules depend on abstractions that low-level modules implement, rather than high-level modules depending directly on low-level concrete classes.

## Tricky Questions

- Q: Does SRP mean "a class should have only one method"? → A: No — SRP is about having one *reason to change* (one responsibility/actor it serves), not a method count; a class can have many methods that all serve one cohesive responsibility.
- Q: Is `Square` extends `Rectangle` always an LSP violation? → A: It's a violation specifically when `Rectangle`'s contract implies independently settable width/height — if the contract were different (immutable shapes with only `area()`), the relationship could be modeled safely; LSP violations are about behavioral *contracts*, not the inheritance keyword itself.

## Coding Exercise

**L1:** Take a class violating SRP (mixing validation + persistence + email-sending) and split it into 3 focused classes. **L2:** Add a new `DiscountPolicy` implementation to the OCP example without touching any existing class. **L3:** Fix the `Square / Rectangle` LSP violation by redesigning around a common `Shape` abstraction instead of inheritance between them. **L4 (Interview):** Explain all 5 SOLID principles with one sentence and one code smell each, from memory. **L5 (Senior):** Refactor a `BadOrderService`-style class (hardcoded concrete dependency) into a DIP-compliant design with constructor injection. **L6 (Mastery):** Given an unfamiliar 100-line class, identify which SOLID principles it likely violates just from its method names and responsibilities (a real senior-review skill).

---

## CHAPTER 14 — Coupling & Cohesion + Mini Project

---

### Telugu Explanation

**Coupling** అంటే ఒక class మరో class మీద ఎంత ఆధారపడి ఉందో (dependency degree) — **Low Coupling** better (classes independent గా change అవ్వచ్చు). **Cohesion** అంటే ఒక class లోపల methods/fields ఎంత closely related గా ఒకే purpose కోసం పనిచేస్తున్నాయో — **High Cohesion** better (class focused గా ఒక్క బాధ్యత కలిగి ఉంటుంది).

### Professional English Explanation

**Coupling** measures how strongly one class depends on another's internals — low coupling means classes can change independently. **Cohesion** measures how tightly a class's own responsibilities relate to one purpose — high cohesion means a class does one thing well. Good OOP design aims for **low coupling, high cohesion** — and SOLID (Ch.13) is essentially a toolkit for achieving exactly that.

## Java Code — Before/After

```
// HIGH COUPLING + LOW COHESION: OrderProcessor knows too much about unrelated concerns
class BadOrderProcessor {
    void process(String item, double price, String email) {
        System.out.println("Validating item: " + item);
        double tax = price * 0.18; // pricing concern
        double total = price + tax;
        System.out.println("Saving order to MySQL directly..."); // persistence concern,
        tightly coupled
        System.out.println("Sending email via SMTP to " + email); // notification concern,
        tightly coupled
        System.out.println("Total: " + total);
    }
}

// LOW COUPLING + HIGH COHESION: each class has one focused responsibility, connected via
// abstractions
interface NotificationService { void notify(String to, String message); }
class EmailNotificationService implements NotificationService {
    public void notify(String to, String message) { System.out.println("Email to " + to + ": "
+ message); }
}

class PricingService {
    double calculateTotal(double price) { return price + (price * 0.18); } // one job:
    pricing
}

class OrderRepository {
    void save(String item, double total) { System.out.println("Order saved: " + item + " => " +
total); } // one job: persistence
}

class OrderProcessor { // orchestrates, doesn't implement
    details itself
    private final PricingService pricingService;
    private final OrderRepository orderRepository;
    private final NotificationService notificationService;

    OrderProcessor(PricingService p, OrderRepository r, NotificationService n) {
        this.pricingService = p; this.orderRepository = r; this.notificationService = n;
    }

    void process(String item, double price, String email) {
        double total = pricingService.calculateTotal(price);
        orderRepository.save(item, total);
        notificationService.notify(email, "Your order for " + item + " totals " + total);
    }
}

public class CouplingCohesionDemo {
    public static void main(String[] args) {
        OrderProcessor processor = new OrderProcessor(
            new PricingService(), new OrderRepository(), new EmailNotificationService());
        processor.process("Laptop", 50000.0, "user@example.com");
    }
}
```

## Output

```
Order saved: Laptop => 59000.0
Email to user@example.com: Your order for Laptop totals 59000.0
```

## Internal Working

- `BadOrderProcessor` is **low cohesion** (pricing + persistence + notification jammed into one method) and **high coupling** (hardcoded to "MySQL" and "SMTP" conceptually — nothing is swappable or independently testable).
- The refactored `OrderProcessor` is **high cohesion** (its only job is orchestration — calling the right collaborators in the right order) and **low coupling** (it depends on interfaces/focused classes, injected via the constructor — directly applying DIP from Ch.13, and previewing Spring's dependency injection style from Book 11).

## Real-World Example

Telugu: Real backend services లో `OrderService`, `PaymentService`, `NotificationService` వేరు వేరు గా design చేయడం — ఒక్క service మారితే మిగతావి affect అవ్వకూడదు. ఇదే low coupling, high cohesion యొక్క production value. English: This orchestration pattern — a coordinating service depending on small, focused, interface-backed collaborators — is exactly the shape of well-designed Spring `@Service` classes (Book 11/12); low coupling and high cohesion are the two qualities code reviews check for most often in real teams.

## Interview Answer

"Coupling measures interdependence between classes — low coupling means classes can change independently without rippling changes elsewhere. Cohesion measures how focused a single class's responsibilities are — high cohesion means a class does one job well. Good design aims for low coupling and high cohesion, which SOLID principles directly help achieve."

## Cross Questions

- Q: Can a class have high cohesion but still be tightly coupled to another class? → A: Yes — e.g., a very focused `EmailSender` class that's still hardcoded to one concrete SMTP library with no abstraction is highly cohesive internally but tightly coupled externally.
- Q: How does dependency injection (Ch.13's DIP) reduce coupling? → A: By depending on interfaces rather than concrete classes and receiving implementations from outside, a class becomes agnostic to *which* concrete implementation it's wired to, drastically lowering coupling.
- Q: Is zero coupling achievable or desirable? → A: No — classes must collaborate to do anything useful; the goal is *minimizing unnecessary* coupling (especially to concrete implementation details), not eliminating all dependencies.

## Tricky Questions

- Q: Can splitting one class into many smaller classes ever *increase* harmful coupling? → A: Yes — if the split is done poorly (e.g., splitting by arbitrary lines rather than true responsibilities), the

resulting classes may need to share so much internal detail that coupling actually increases; cohesion-driven splitting (one true responsibility per class) avoids this.

## Coding Exercise

**L1:** Identify coupling/cohesion problems in a given "God class" (provided or self-written) mixing 3+ unrelated responsibilities. **L2:** Refactor that class into 2–3 focused, loosely-coupled classes connected via constructor injection. **L3:** Measure "coupling" informally by counting concrete class references vs interface references before/after your refactor. **L4 (Interview):** Explain, with your own example, why "low coupling, high cohesion" is often summarized as the *goal* that SOLID's five principles serve. **L5 (Senior):** Review the `OrderProcessor` example and propose how you'd unit-test it in isolation (mocking collaborators) — a direct preview of Book 15's testing techniques enabled by this design. **L6 (Mastery):** Explain coupling and cohesion from memory using two contrasting example classes you invent on the spot.

---



## Mini Project: Library Management (Console, OOP-Only)

### Goal

Apply every pillar from this book in one system — no Collections Framework internals beyond `List / ArrayList` usage (full mastery in Book 05), no exceptions beyond what Book 01 already covered informally.

### Requirements

1. Abstract class `LibraryItem` (fields: `title`, `id`; abstract method `getLoanPeriodDays()`; concrete `describe()`).
2. Subclasses `Book` extends `LibraryItem` (14-day loan) and `DVD` extends `LibraryItem` (7-day loan) — demonstrates inheritance + overriding.
3. Interface `Reservable` { `void reserve(String memberName);` }, implemented only by `Book` (DVDs can't be reserved in this system) — demonstrates interface segregation (Ch.13) in practice.
4. `Library` class (composition: owns a `List<LibraryItem>` it manages internally) with methods `addItem()`, `findById()`, `printCatalog()`.
5. `Member` class in an **association** relationship with `Library` (a `Member` knows which `Library` they belong to, but neither owns the other's lifecycle).
6. Apply SRP: keep loan-period logic, catalog management, and reservation logic in their own methods/classes — don't mix them into one giant class.

### Concepts Reinforced

Abstraction (Ch.3, 9) · Inheritance + overriding (Ch.4, 8) · Interfaces + ISP (Ch.10, 13) · Composition vs Association (Ch.12) · SRP/OCP/DIP (Ch.13) · Coupling/Cohesion (Ch.14).

## Stretch Goals

- Add a `MagazineItem` type without modifying `Library`'s existing code (prove OCP).
  - Add a `NotificationService` interface (email/SMS) that `Library` depends on abstractly, injected via constructor (prove DIP).
-

## FINAL REVISION NOTES

---

- **Encapsulation** ⇒ "getter/setter for every field" — it's about protecting invariants via validated behavior.
  - **Abstraction** hides complexity from *callers*; **Encapsulation** hides state from *misuse*. Different problems, often confused.
  - **Inheritance** = IS-A, single-class only in Java; **Interfaces** = multiple "CAN-DO" capabilities.
  - **super** : constructor chaining is always top-down through the hierarchy; `super.method()` extends rather than replaces behavior.
  - **Overloading** = compile-time, resolved by argument types, priority: exact → widening → autoboxing → varargs.
  - **Overriding** = runtime, resolved by actual object type (dynamic dispatch); `static / private / final` methods are excluded.
  - **Abstract class**: state + partial implementation + single inheritance. **Interface**: pure contract + multiple implementation (plus `default` methods since Java 8, Book 07).
  - **IS-A vs HAS-A**: HAS-A splits into Composition (strong, owns lifecycle), Aggregation (weak, independent lifecycle), Association (no ownership).
  - **SOLID**: SRP (one reason to change), OCP (extend without modifying), LSP (subtypes must honor the parent's contract), ISP (small focused interfaces), DIP (depend on abstractions).
  - **Coupling/Cohesion**: aim for low coupling (interfaces + injection), high cohesion (one class, one job) — SOLID is the toolkit to get there.
-



# CHEAT SHEET

---

IS-A = inheritance (extends) | HAS-A = composition/aggregation/association (field reference)  
Overloading: same name, diff params, compile-time, no return-type-only overloads  
Overriding: same signature, runtime dispatch, can't override static/private/final, can't narrow access or widen checked exceptions, covariant return types allowed  
Abstract class: state + ctor + partial impl, single extends, cannot instantiate  
Interface: contract (+default/static since Java 8), multiple implements, fields are public static final constants  
Composition: container CREATES the part (strong lifecycle bind)  
Aggregation: container RECEIVES the part (independent lifecycle)  
Association: plain reference, no ownership either direction  
SOLID: S-one responsibility, O-extend don't modify, L-subtypes honor contracts, I-small interfaces, D-depend on abstractions  
Coupling: interdependence (want LOW) | Cohesion: focus of one class (want HIGH)



# INTERVIEW QUESTION BANK — Java OOPS

---

## Beginner

1. What are the four pillars of OOP? Define each in one line.
2. Difference between method overloading and overriding?
3. What is the difference between an abstract class and an interface?
4. What is `super` used for?
5. What is the difference between IS-A and HAS-A?

**Intermediate** 6. Why does Java not support multiple inheritance for classes? 7. Explain dynamic method dispatch with an example. 8. What is the difference between composition and aggregation? Give an example of each. 9. Explain SRP and OCP with a code example each. 10. Why can't `static` methods be overridden?

**Advanced** 11. Explain the Liskov Substitution Principle with a real violation example (e.g., Square/Rectangle) and how to fix it. 12. When would you choose an abstract class over an interface, given Java 8+ default methods? 13. Explain Java's overload resolution priority order (exact → widening → boxing → varargs) with an example that would be ambiguous otherwise. 14. What happens if a subclass constructor doesn't call `super(...)` explicitly, and the superclass has no no-arg constructor? 15. Explain the Dependency Inversion Principle and how it enables frameworks like Spring to work.

**Senior/Architect** 16. Review a class violating 3 SOLID principles simultaneously and redesign it, explaining each fix. 17. Design a small plugin architecture (e.g., payment gateways) using interfaces + DIP that supports adding new implementations with zero changes to existing code. 18. Explain how low coupling and high cohesion trade off against each other when splitting a large class, and how to avoid over-splitting. 19. Walk through exactly what happens (dynamic dispatch, memory) when a polymorphic call is made through a superclass reference to an overridden method. 20. Design a domain model combining IS-A and multiple HAS-A relationships (composition + aggregation + association) for a real system (e.g., a hospital: `Doctor`, `Patient`, `Department`, `Appointment`).

*(Full short/professional/deep-senior answer scaffolding for every question across the entire series lives in Book 23 — Java Interview Master Book.)*

---



## CROSS-QUESTION ENGINE — Sample Chains

---

**Q: What is polymorphism?** → Q: What's the difference between compile-time and runtime polymorphism? → Q: How does the JVM resolve an overridden method call at runtime? → Q: Why can't fields be polymorphic? → Q: Why can't static methods be overridden?

**Q: What is an interface?** → Q: Can it have fields? → Q: Can it have method bodies? → Q: How does that differ pre- and post-Java 8? → Q: When would you choose it over an abstract class?

**Q: What is SOLID?** → Q: Give an example of an SRP violation. → Q: How does OCP relate to polymorphism? → Q: What's a real LSP violation you've seen or can construct? → Q: How does DIP enable dependency injection frameworks like Spring?

---



## CONSOLIDATED EXERCISES (All Levels)

---

**L1 — Beginner:** Redo each chapter's L1 exercise unaided. **L2 — Intermediate:** Redo each chapter's L2/L3 exercises, explaining each design choice out loud in Telugu. **L3 — Advanced:** Build a 3-level inheritance hierarchy combining an abstract class, two interfaces, and at least one of each HAS-A relationship type. **L4 — Interview:** Answer all 20 Interview Bank questions from memory, under 90 seconds each. **L5 — Senior:** Complete the Library Management mini project fully, including both stretch goals (new item type via OCP, injected `NotificationService` via DIP). **L6 — Mastery:** Teach Chapters 6 (polymorphism), 8 (dynamic dispatch), and 13 (SOLID) out loud, drawing every diagram from scratch, no notes.

---



## ONE-DAY REVISION PLAN (≈5 hours)

| Time      | Focus                                                                                                             |
|-----------|-------------------------------------------------------------------------------------------------------------------|
| 0:00–0:30 | Ch.1–2: Four pillars overview + Encapsulation deep dive                                                           |
| 0:30–1:00 | Ch.3: Abstraction — re-read the payment processor example twice                                                   |
| 1:00–1:45 | Ch.4–5: Inheritance + <code>super</code> /constructor chaining — redraw the constructor-order diagram from memory |
| 1:45–2:00 | Break                                                                                                             |
| 2:00–3:00 | Ch.6–8: Polymorphism, Overloading, Overriding — the highest-density interview block, don't rush                   |
| 3:00–3:30 | Ch.9–11: Abstract class vs Interface — memorize the decision table                                                |
| 3:30–4:00 | Ch.12: IS-A vs HAS-A — practice classifying 5 fresh examples                                                      |
| 4:00–4:45 | Ch.13–14: SOLID + Coupling/Cohesion — walk through every bad/good code pair                                       |
| 4:45–5:00 | Answer the full Interview Question Bank from memory                                                               |



# ONE-WEEK MASTER REVISION PLAN

| Day | Focus                                                                                                                                        |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | Ch.1–3 (pillars overview, encapsulation, abstraction) — code every example yourself                                                          |
| 2   | Ch.4–5 (inheritance, super/chaining) — build your own 3-level hierarchy                                                                      |
| 3   | Ch.6–8 (polymorphism, overloading, overriding) — this is the core interview battleground; do all 6 exercise levels                           |
| 4   | Ch.9–11 (abstract class, interface, decision framework) — build the combined <code>Person / Payable / Contractor</code> example from scratch |
| 5   | Ch.12–13 (IS-A/HAS-A, SOLID) — rewrite all 5 SOLID bad/good pairs without looking                                                            |
| 6   | Ch.14 + Mini Project — build the full Library Management system plus both stretch goals                                                      |
| 7   | Full mock interview: all 20 bank questions + all cross-question chains, in Telugu and English, without notes                                 |



## FINAL MASTERY CHECKLIST

---

- ☐ I can define and give a code example for each of the four OOP pillars.
- ☐ I can explain why "getter/setter for every field" isn't automatically good encapsulation.
- ☐ I can explain the difference between abstraction and encapsulation precisely.
- ☐ I can explain constructor chaining order across a 3+ level hierarchy from memory.
- ☐ I can explain Java's overload resolution priority order exactly.
- ☐ I can explain dynamic method dispatch and why static/private/final methods are excluded.
- ☐ I can list all 5 overriding rules and justify each.
- ☐ I can decide abstract class vs interface for a new design in under 30 seconds, with justification.
- ☐ I can classify a relationship as IS-A, Composition, Aggregation, or Association correctly.
- ☐ I can state all 5 SOLID principles and give a bad/good code example for each, unaided.
- ☐ I can explain coupling and cohesion, and why SOLID exists to serve those two goals.
- ☐ I built the Library Management mini project, including both stretch goals (OCP + DIP).
- ☐ I answered the full Interview Question Bank confidently in both Telugu and English.

**Once every box is checked, you are ready for `03_JVM_Memory_Management.md`.**